

Algoritmos Lineares: Counting Sort e Radix Sort

Os algoritmos **Counting Sort** e **Radix Sort** são chamados de **algoritmos lineares** porque, em determinadas condições, conseguem ordenar elementos em tempo próximo de **$O(n)$** , sendo mais rápidos que algoritmos de comparação como Quick Sort, Merge Sort e Heap Sort (que normalmente possuem complexidade $O(n \log n)$).

Counting Sort

1. Counting Sort

Conceito

O **Counting Sort** não compara elementos entre si.

Ele funciona contando quantas vezes cada valor aparece no vetor e depois reconstruindo o vetor ordenado.

É ideal quando:

- Os valores são inteiros.
- O maior valor não é muito grande.
- Existe uma faixa conhecida de valores.

Counting Sort

Exemplo

Vetor:

[4, 2, 2, 8, 3, 3, 1]

Passo 1: Encontrar o maior elemento

Maior valor:

8

Criamos um vetor de contagem com posições de 0 a 8.

Índice: 0 1 2 3 4 5 6 7 8

Contagem: 0 0 0 0 0 0 0 0 0

Counting Sort

Passo 2: Contar ocorrências

Percorremos o vetor original:

4 → conta[4]++

2 → conta[2]++

2 → conta[2]++

8 → conta[8]++

3 → conta[3]++

3 → conta[3]++

1 → conta[1]++

Resultado:

Índice: 0 1 2 3 4 5 6 7 8

Contagem: 0 1 2 2 1 0 0 0 1

Counting Sort

Passo 3: Reconstruir o vetor

Lemos o vetor de contagem:

1 aparece 1 vez

2 aparece 2 vezes

3 aparece 2 vezes

4 aparece 1 vez

8 aparece 1 vez

Resultado:

[1, 2, 2, 3, 3, 4, 8]

COUNTING SORT – PASSO A PASSO

VETOR ORIGINAL:

4

2

2

8

3

3

1

1

ENCONTRAR O MAIOR ELEMENTO

Maior elemento do vetor = 8

Vamos criar um vetor de contagem com índices de 0 até 8.

ÍNDICE

0

1

2

3

4

5

6

8

CONTAGEM

0

0

0

0

0

0

0

0

2

CONTAR AS OCORRÊNCIAS DE CADA ELEMENTO

Percorremos o vetor original e incrementamos a contagem da posição correspondente a cada valor.

ÍNDICE

0

1

2

3

4

5

6

8

CONTAGEM

0

1

2

2

1

0

0

1

1 aparece
1 vez

2 aparece
2 vezes

3 aparece
2 vezes

4 aparece
1 vez

8 aparece
1 vez

3

RECONSTRUIR O VETOR ORDENADO

Percorremos o vetor de contagem da esquerda para a direita. Para cada índice, colocamos no vetor original o índice tantas vezes quanto sua contagem indica.

ÍNDICE (i)

0

1

2

3

4

5

6

7

8

CONTAGEM[i]

0

1

2

2

1

0

0

0

1

AÇÃO

nada

colocar 1
1 vez

colocar 2
2 vezes

colocar 3
2 vezes

colocar 4
1 vez

nada

nada

nada

colocar 8
1 vez

VETOR EM
CONSTRUÇÃO

- - - - -

1 - - - - -

1 2 2 - - - -

1 2 2 3 3 - -

1 2 2 3 3 4 -

1 2 2 3 3 4 8

4

VETOR ORDENADO (RESULTADO FINAL)

1

2

2

3

3

4

8



OBSERVAÇÕES:

- O Counting Sort é um algoritmo estável.
- Ele não compara elementos, apenas conta quantas vezes cada valor aparece.
- Complexidade:** $O(n + k)$, onde n é a quantidade de elementos e k é o maior valor do vetor (neste caso, 8).

Counting Sort

```
#include <stdio.h>
```

```
int getMax(int vetor[], int n) {  
    int max = vetor[0];  
    for(int i = 1; i < n; i++)  
        if(vetor[i] > max)  
            max = vetor[i];  
    return max;  
}
```


Counting Sort

```
void countingSort(int vetor[], int n, int exp) {  
    int saida[n];  
    int contagem[10] = {0};  
    // Contagem dos dígitos  
    for(int i = 0; i < n; i++)  
        contagem[(vetor[i] / exp) % 10]++;  
    // Soma acumulada  
    for(int i = 1; i < 10; i++)  
        contagem[i] += contagem[i - 1];  
    // Construção do vetor ordenado  
    for(int i = n - 1; i >= 0; i--) {  
        int indice = (vetor[i] / exp) % 10;  
        saida[contagem[indice] - 1] = vetor[i];  
        contagem[indice]--;  
    }  
    // Copiar para o vetor original  
    for(int i = 0; i < n; i++)  
        vetor[i] = saida[i];  
}
```


Counting Sort

```
int main() {  
    int vetor[] = {4, 2, 2, 8, 3, 3, 1};  
  
    int n = sizeof(vetor)/sizeof(vetor[0]);  
  
    countingSort(vetor, n);  
  
    printf("Vetor ordenado:\n");  
  
    for(int i = 0; i < n; i++)  
        printf("%d ", vetor[i]);  
  
    return 0;  
}
```

Radix Sort

2. Radix Sort

Conceito

O **Radix Sort** ordena os números dígito por dígito.

Ele normalmente utiliza o **Counting Sort** como algoritmo auxiliar.

A ordenação começa pelo:

Dígito das unidades

Dígito das dezenas

Dígito das centenas

E assim por diante

Exemplo

Vetor:

[170, 45, 75, 90, 802, 24, 2, 66]

Radix Sort

Passo 1: Ordenar pelas unidades

170 → 0

45 → 5

75 → 5

90 → 0

802 → 2

24 → 4

2 → 2

66 → 6

Resultado:

[170, 90, 802, 2, 24, 45, 75, 66]

Radix Sort

Passo 2: Ordenar pelas dezenas

170 → 7

90 → 9

802 → 0

2 → 0

24 → 2

45 → 4

75 → 7

66 → 6

Resultado:

[802, 2, 24, 45, 66, 170, 75, 90]

Radix Sort

Passo 3: Ordenar pelas centenas

802 → 8

2 → 0

24 → 0

45 → 0

66 → 0

170 → 1

75 → 0

90 → 0

Resultado final:

[2, 24, 45, 66, 75, 90, 170, 802]

Radix Sort

Como o Radix Sort funciona visualmente

Vetor Original

170 45 75 90 802 24 2 66

↓ Unidades

170 90 802 2 24 45 75 66

↓ Dezenas

802 2 24 45 66 170 75 90

↓ Centenas

2 24 45 66 75 90 170 802

VETOR ORIGINAL

170	45	75	90	802	24	2	66
-----	----	----	----	-----	----	---	----

Vamos ordenar da direita para a esquerda (unidades, dezenas, centenas) usando Counting Sort em cada etapa.

1ª PASSAGEM – ORDENAR PELO DÍGITO DAS UNIDADES

Vetores e dígitos (unidades)	170	45	75	90	802	24	2	66
	0	5	5	0	2	4	2	6

Dígito das unidades	0	5	5	0	2	4	2	6
---------------------	---	---	---	---	---	---	---	---

↓ Após ordenar pelo dígito das unidades:

	170	90	802	2	24	45	75	66
Unidades	0	0	2	2	4	5	5	6

2ª PASSAGEM – ORDENAR PELO DÍGITO DAS DEZENAS

Vetores e dígitos (dezenas)	170	90	802	2	24	45	75	66
	7	9	0	0	2	4	7	6

Dígito das dezenas	7	9	0	0	2	4	7	6
--------------------	---	---	---	---	---	---	---	---

↓ Após ordenar pelo dígito das dezenas:

	802	2	24	45	66	170	75	90
Dezenas	0	0	2	4	6	7	7	9

3ª PASSAGEM – ORDENAR PELO DÍGITO DAS CENTENAS

Vetores e dígitos (centenas)	802	2	24	45	66	170	75	90
	8	0	0	0	0	1	0	0

Dígito das centenas	8	0	0	0	0	1	0	0
---------------------	---	---	---	---	---	---	---	---

↓ Após ordenar pelo dígito das centenas:

	2	24	45	66	75	90	170	802
Centenas	0	0	0	0	0	0	1	8

VETOR FINAL ORDENADO

2	24	45	66	75	90	170	802
---	----	----	----	----	----	-----	-----



Resumo: O Radix Sort ordena os números dígito por dígito, da direita para a esquerda. A cada passagem, os números ficam ordenados com base naquele dígito, mas a ordenação final só está correta após a última passagem (centenas, no exemplo).

Radix Sort

```
#include <stdio.h>
```

```
int getMax(int vetor[], int n) {  
    int max = vetor[0];  
    for(int i = 1; i < n; i++)  
        if(vetor[i] > max)  
            max = vetor[i];  
    return max;  
}
```

Radix Sort

```
void radixSort(int vetor[], int n) {  
    int max = getMax(vetor, n);  
  
    for(int exp = 1; max / exp > 0; exp *= 10)  
        countingSort(vetor, n, exp);  
}  
  
int main() {  
    int vetor[] = {170, 45, 75, 90, 802, 24, 2, 66};  
    int n = sizeof(vetor)/sizeof(vetor[0]);  
    radixSort(vetor, n);  
    printf("Vetor ordenado:\n");  
    for(int i = 0; i < n; i++)  
        printf("%d ", vetor[i]);  
    return 0;  
}
```

Algoritmos Lineares: Counting Sort e Radix Sort

Resumo Comparativo

Característica	Counting Sort	Radix Sort
Compara elementos?	Não	Não
Baseado em contagem?	Sim	Sim
Usa Counting Sort?	Não	Sim
Funciona melhor com	Faixa pequena de valores	Números inteiros com vários dígitos
Complexidade	$O(n + k)$	$O(d \times (n + k))$
Estável	Sim	Sim (quando usa Counting Sort estável)

Matriz e lista de adjacência(GRAFOS)

Quando trabalhamos com **grafos**, precisamos de uma forma de armazenar os vértices e as conexões (arestas). As duas representações mais utilizadas são:

Matriz de Adjacência

Lista de Adjacência

Matriz e lista de adjacência(GRAFOS)

O que é um Grafo?

Imagine as seguintes cidades conectadas por estradas:

```
graph LR; A --- B; A --- C; C --- D; B --- D
```

A diagram showing four nodes labeled A, B, C, and D arranged in a square. A is at the top-left, B at the top-right, C at the bottom-left, and D at the bottom-right. A dashed line connects A and B. A solid line connects A and C. A dashed line connects C and D. A solid line connects B and D.

Matriz e lista de adjacência(GRAFOS)

Podemos representar isso como:

A -> B

A -> C

B -> D

C -> D

Onde:

A, B, C e D são os **vértices**

As ligações são as **arestas**

Matriz e lista de adjacência(GRAFOS)

1. Matriz de Adjacência

Uma matriz de adjacência é uma tabela onde:

Linhas representam a origem.

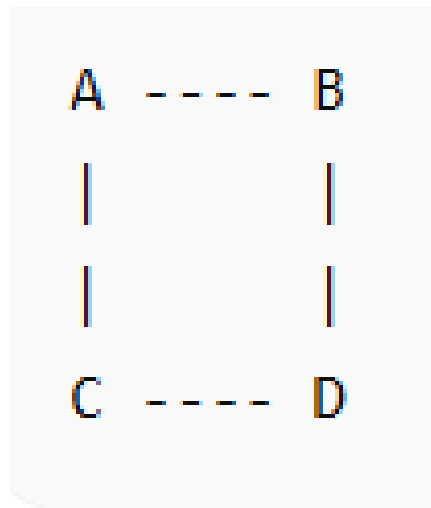
Colunas representam o destino.

Valor 1 significa que existe conexão.

Valor 0 significa que não existe conexão.

Exemplo

Considere o grafo:



Matriz e lista de adjacência(GRAFOS)

Numerando os vértices:

A = 0

B = 1

C = 2

D = 3

A matriz fica:

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

OU:

A B C D

A -> [0 1 1 0]

B -> [1 0 0 1]

C -> [1 0 0 1]

D -> [0 1 1 0]

Matriz e lista de adjacência(GRAFOS)

```
#include <stdio.h>
#define V 4
int main() {
    int grafo[V][V] = {
        {0,1,1,0},
        {1,0,0,1},
        {1,0,0,1},
        {0,1,1,0}
    };
    printf("Matriz de Adjacencia:\n\n");
    for(int i=0;i<V;i++) {
        for(int j=0;j<V;j++) {
            printf("%d ", grafo[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Matriz e lista de adjacência(GRAFOS)

Saída

```
0  1  1  0
1  0  0  1
1  0  0  1
0  1  1  0
```

Vantagens da Matriz

- ✓ Fácil implementação.
- ✓ Verificar se existe uma aresta é muito rápido:

```
if (grafo[0][1] == 1)
```

Matriz e lista de adjacência(GRAFOS)

Desvantagens da Matriz

Se tivermos:

1000 vértices

Precisaremos:

$1000 \times 1000 = 1.000.000$ posições

Mesmo que existam poucas conexões.

Isso desperdiça memória.

Matriz e lista de adjacência(GRAFOS)

2. Lista de Adjacência

Em vez de guardar todas as possíveis conexões, armazenamos apenas as que realmente existem.

Mesmo Grafo

```
A  ----  B
|         |
|         |
C  ----  D
```

Representação:

A -> B -> C

B -> A -> D

C -> A -> D

D -> B -> C

Matriz e lista de adjacência(GRAFOS)

Visualização

0 -> 1 -> 2

1 -> 0 -> 3

2 -> 0 -> 3

3 -> 1 -> 2

Matriz e lista de adjacência(GRAFOS)

```
#include <stdio.h>
#include <stdlib.h>
#define V 4

typedef struct No {
    int vertice;
    struct No* prox;
} No;

No* lista[V];

void adicionarAresta(int origem, int destino) {

    No* novo = (No*) malloc(sizeof(No));
    novo->vertice = destino;
    novo->prox = lista[origem];
    lista[origem] = novo;
}
```

Matriz e lista de adjacência(GRAFOS)

```
void imprimirGrafo() {  
  
    for(int i=0;i<V;i++) {  
  
        printf("%d -> ", i);  
  
        No* temp = lista[i];  
  
        while(temp != NULL) {  
            printf("%d ", temp->vertice);  
            temp = temp->prox;  
        }  
  
        printf("\n");  
    }  
}
```


Matriz e lista de adjacência(GRAFOS)

```
int main() {  
    for(int i=0;i<V;i++)  
        lista[i] = NULL;  
    adicionarAresta(0,1);  
    adicionarAresta(0,2);  
    adicionarAresta(1,0);  
    adicionarAresta(1,3);  
    adicionarAresta(2,0);  
    adicionarAresta(2,3);  
    adicionarAresta(3,1);  
    adicionarAresta(3,2);  
    imprimirGrafo();  
    return 0;  
}
```

Matriz e lista de adjacência(GRAFOS)

Saída

```
0 -> 2 1
1 -> 3 0
2 -> 3 0
3 -> 2 1
```

A ordem pode mudar porque estamos inserindo no início da lista.

Comparação

Característica	Matriz	Lista
Implementação	Simples	Mais complexa
Uso de memória	Alto	Baixo
Verificar aresta	$O(1)$	$O(\text{grau do vértice})$
Percorrer vizinhos	$O(V)$	$O(\text{grau})$
Grafos densos	Melhor	Boa
Grafos esparsos	Ruim	Melhor

Matriz e lista de adjacência(GRAFOS)

Exemplo de Consumo de Memória

Suponha:

1000 vértices

2000 arestas

Matriz

1000 x 1000

= 1.000.000 posições

Lista

Armazena apenas:

2000 arestas

A economia de memória é enorme.

Matriz e lista de adjacência(GRAFOS)

Quando usar cada uma?

Use Matriz de Adjacência quando:

O grafo é pequeno.

Existem muitas conexões.

Você precisa consultar rapidamente se uma aresta existe.

Exemplo:

Redes pequenas

Jogos simples

Exercícios acadêmicos

Matriz e lista de adjacência(GRAFOS)

Use Lista de Adjacência quando:

O grafo é grande.

Existem poucas conexões.

Memória é importante.

Exemplo:

GPS

Google Maps

Redes Sociais

Internet

Algoritmo de Dijkstra com Heap

Algoritmo de Dijkstra

O **Algoritmo de Dijkstra** é um algoritmo utilizado para encontrar o **menor caminho** entre um vértice de origem e todos os outros vértices de um grafo que possua **pesos positivos**.

Algoritmo de Dijkstra

Entendendo de forma simples

Imagine que você está em uma cidade e quer descobrir qual é o caminho mais curto para chegar a vários bairros.

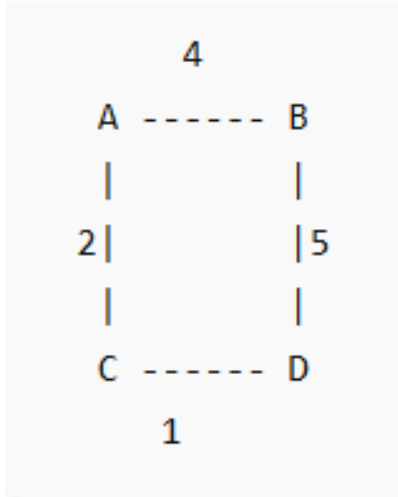
O Dijkstra funciona da seguinte forma:

1. Começa no ponto de origem.
2. Marca a distância da origem para ela mesma como **0**.
3. Marca todas as outras distâncias como **infinito**.
4. Escolhe o vértice não visitado com a menor distância conhecida.
5. Atualiza as distâncias dos seus vizinhos.
6. Repete até visitar todos os vértices.

Algoritmo de Dijkstra

Exemplo

Considere o seguinte grafo:



E uma ligação adicional:

A ----- 7 ----- D

Algoritmo de Dijkstra

Representação completa:

A -> B = 4

A -> C = 2

A -> D = 7

B -> D = 5

C -> D = 1

Queremos descobrir o menor caminho saindo de **A**.

Algoritmo de Dijkstra

Passo 1

Distâncias iniciais:

Vértice	Distância
A	0
B	∞
C	∞
D	∞

Visitados: nenhum.

Algoritmo de Dijkstra

Passo 2

Escolhemos A (distância 0).

Analisamos seus vizinhos:

A → B = 4

A → C = 2

A → D = 7

Atualizamos:

Vértice	Distância
A	0
B	4
C	2
D	7

A torna-se visitado.

Algoritmo de Dijkstra

Passo 3

O menor não visitado é C (2).

Analizamos:

$$C \rightarrow D = 1$$

Novo custo até D:

$$A \rightarrow C \rightarrow D$$

$$2 + 1 = 3$$

Algoritmo de Dijkstra

Como $3 < 7$:
Atualizamos D.

Vértice	Distância
A	0
B	4
C	2
D	3

C visitado.

Algoritmo de Dijkstra

Passo 4

O menor não visitado é D (3).

D não melhora nenhum caminho.

D visitado.

Algoritmo de Dijkstra

Passo 5

Resta B (4).

$B \rightarrow D = 5$

$4 + 5 = 9$

Como $9 > 3$, não atualizamos.

B visitado.

Algoritmo de Dijkstra

Resultado Final

Menores distâncias a partir de A:

Destino	Distância
A	0
B	4
C	2
D	3

Caminhos:

$A \rightarrow B = 4$

$A \rightarrow C = 2$

$A \rightarrow C \rightarrow D = 3$

Algoritmo de Dijkstra

```
#include <stdio.h>
#include <limits.h>
```

```
#define V 4
```

```
//== Para simplificar, utilizaremos uma matriz de adjacência.
```

```
int minDistancia(int dist[], int visitado[]) {
    int min = INT_MAX;
    int minIndice;

    for (int v = 0; v < V; v++) {
        if (!visitado[v] && dist[v] <= min) {
            min = dist[v];
            minIndice = v;
        }
    }

    return minIndice;
}
```

```

void dijkstra(int grafo[V][V], int origem) {
    int dist[V];
    int visitado[V];
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visitado[i] = 0;
    }
    dist[origem] = 0;
    for (int cont = 0; cont < V - 1; cont++) {
        int u = minDistancia(dist, visitado);
        visitado[u] = 1;
        for (int v = 0; v < V; v++) {
            if (!visitado[v] &&
                grafo[u][v] &&
                dist[u] != INT_MAX &&
                dist[u] + grafo[u][v] < dist[v]) {

                dist[v] = dist[u] + grafo[u][v];
            }
        }
    }
    printf("Menores distancias a partir da origem:\n");
    for (int i = 0; i < V; i++) {
        printf("Vertice %d -> %d\n", i, dist[i]);
    }
}

```

Algoritmo de Dijkstra

```
int main() {  
  
    int grafo[V][V] = {  
        {0, 4, 2, 7},  
        {4, 0, 0, 5},  
        {2, 0, 0, 1},  
        {7, 5, 1, 0}  
    };  
  
    dijkstra(grafo, 0);  
  
    return 0;  
}
```

Algoritmo de Dijkstra

Algoritmos de travessia de grafos

Os algoritmos de travessia de grafos são utilizados para visitar todos os vértices de um grafo de forma organizada.

Algoritmos de travessia de grafos

Os dois algoritmos mais conhecidos são:

Busca em Largura (BFS - Breadth First Search)

Busca em Profundidade (DFS - Depth First Search)

Uma analogia simples:

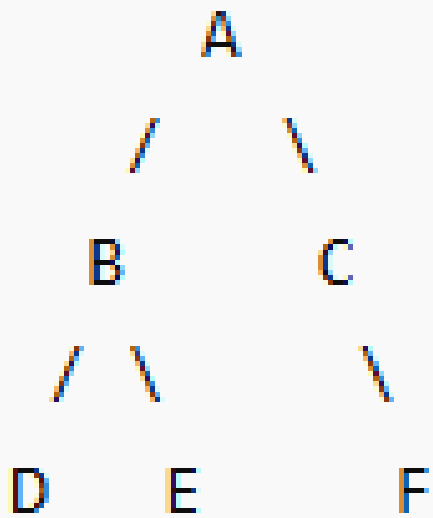
BFS → visita primeiro os vizinhos mais próximos.

DFS → segue um caminho até o fim antes de voltar.

Algoritmos de travessia de grafos

Grafo de Exemplo

Vamos utilizar o seguinte grafo:



Algoritmos de travessia de grafos

Representando os vértices por números:

A = 0

B = 1

C = 2

D = 3

E = 4

F = 5

Lista de adjacência:

0 -> 1 -> 2

1 -> 3 -> 4

2 -> 5

3

4

5

Algoritmos de travessia de grafos

1. Busca em Largura (BFS)

A BFS visita os vértices por níveis.

Imagine uma pedra caindo em um lago:

Nível 0: A

Nível 1: B C

Nível 2: D E F

A ordem de visita será:

A → B → C → D → E → F

Algoritmos de travessia de grafos

Como a BFS funciona

Ela utiliza uma **fila (Queue)**.

Passo 1

Fila:

[A]

Visita:

A

Algoritmos de travessia de grafos

Passo 2

Adiciona os vizinhos de A:

[B, C]

Visita:

$A \rightarrow B$

Algoritmos de travessia de grafos

Passo 3

Remove B e adiciona seus vizinhos:

[C, D, E]

Visita:

$A \rightarrow B \rightarrow C$

Algoritmos de travessia de grafos

Passo 4

Remove C e adiciona F:

[D, E, F]

Visita:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

Algoritmos de travessia de grafos

```
#include <stdio.h>
#define V 6
void BFS(int grafo[V][V], int inicio) {
    int visitado[V] = {0};
    int fila[V];
    int inicioFila = 0;
    int fimFila = 0;
    visitado[inicio] = 1;
    fila[fimFila++] = inicio;
    while (inicioFila < fimFila) {
        int atual = fila[inicioFila++];
        printf("%d ", atual);
        for (int i = 0; i < V; i++) {
            if (grafo[atual][i] == 1 &&
                !visitado[i]) {
                visitado[i] = 1;
                fila[fimFila++] = i;
            }
        }
    }
}
```

Algoritmos de travessia de grafos

```
int main() {  
  
    int grafo[V][V] = {  
  
        //A B C D E F|  
        {0,1,1,0,0,0}, // A  
        {0,0,0,1,1,0}, // B  
        {0,0,0,0,0,1}, // C  
        {0,0,0,0,0,0}, // D  
        {0,0,0,0,0,0}, // E  
        {0,0,0,0,0,0}  // F  
    };  
  
    printf("BFS: ");  
    BFS(grafo,0);  
  
    return 0;  
}
```

Algoritmos de travessia de grafos

```
int main() {  
  
    int grafo[V][V] = {  
  
        //A B C D E F|  
        {0,1,1,0,0,0}, // A  
        {0,0,0,1,1,0}, // B  
        {0,0,0,0,0,1}, // C  
        {0,0,0,0,0,0}, // D  
        {0,0,0,0,0,0}, // E  
        {0,0,0,0,0,0}  // F  
    };  
  
    printf("BFS: ");  
    BFS(grafo,0);  
  
    return 0;  
}
```

Algoritmos de travessia de grafos

2. Busca em Profundidade (DFS)

A DFS tenta ir o mais fundo possível antes de voltar.

Pense em um labirinto:

Se existe um corredor,
continue andando.

Só volte quando encontrar
um beco sem saída.

Funcionamento

Começando em A:

A

↓

B

↓

D

Algoritmos de travessia de grafos

D não possui filhos.

Volta para B.

A

↓

B

↓

E

E termina.

Volta para A.

A

↓

C

↓

F

Ordem de visita

A → B → D → E → C → F

ou

0 → 1 → 3 → 4 → 2 → 5

Algoritmos de travessia de grafos

```
void DFS(int grafo[V][V],
        int vertice,
        int visitado[]) {

    visitado[vertice] = 1;

    printf("%d ", vertice);

    for(int i = 0; i < V; i++) {

        if(grafo[vertice][i] == 1 &&
            !visitado[i]) {

            DFS(grafo, i, visitado);

        }

    }

}
```

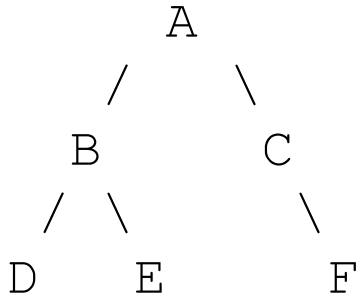
Algoritmos de travessia de grafos

```
int main() {  
  
    int grafo[V][V] = {  
  
        //A B C D E F  
        {0,1,1,0,0,0},  
        {0,0,0,1,1,0},  
        {0,0,0,0,0,1},  
        {0,0,0,0,0,0},  
        {0,0,0,0,0,0},  
        {0,0,0,0,0,0}  
    };  
  
    int visitado[V] = {0};  
  
    printf("DFS: ");  
  
    DFS(grafo,0,visitado);  
  
    return 0;  
}
```

Algoritmos de travessia de grafos

Visualizando a Diferença

Considere novamente:



BFS

Visita por níveis:

A

B C

D E F

Resultado:

A B C D E F

Algoritmos de travessia de grafos

DFS

Vai até o fundo primeiro:

A

↓

B

↓

D

(volta)

E

(volta)

C

↓

F

Resultado:

A B D E C F

Algoritmos de travessia de grafos

Estruturas Utilizadas

Algoritmo

Estrutura

BFS

Fila (Queue)

DFS

Pilha (Stack) ou Recursão

Algoritmos de travessia de grafos

Aplicações Práticas

BFS

- Encontrar o caminho mais curto em grafos sem peso.
- GPS simples.
- Redes sociais.
- Jogos.

Exemplo:

Qual o menor número de amizades entre João e Maria?

DFS

- Detectar ciclos.
- Resolver labirintos.
- Ordenação topológica.
- Encontrar componentes conexos.

Exemplo:

Existe algum caminho entre duas páginas da web?

Algoritmos de travessia de grafos

Resumo

Característica	BFS	DFS
Estratégia	Visita por níveis	Vai até o fundo
Estrutura	Fila	Pilha/Recursão
Ordem no exemplo	A B C D E F	A B D E C F
Menor caminho sem peso	Sim	Não
Uso comum	Rotas e distâncias	Exploração e análise

Regra fácil para decorar

- BFS (Busca em Largura)** → "Visita os irmãos primeiro".
- DFS (Busca em Profundidade)** → "Visita os filhos primeiro".

Essa é a principal diferença entre os dois algoritmos de travessia de grafos.